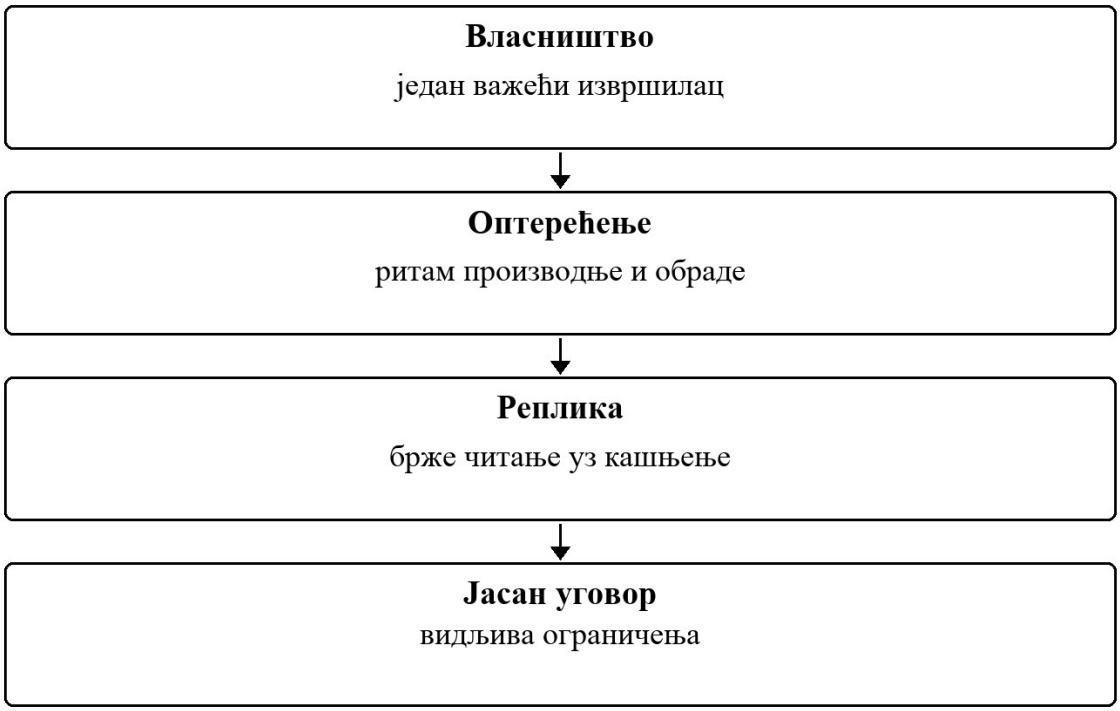
Координација, проток и евентуална конзистентност

Основе дистрибуираног програмирања · 08

Напредни дистрибуирани дизајн прихвата да различите компоненте могу привремено имати различит поглед на стање. Изазов није да се та чињеница сакрије, већ да се власништво, пребацивање рада, оптерећење и свежина података моделују тако да корисник и оператер знају шта систем може да гарантује.

Координација спречава сукоб над ексклузивним радом, повратни притисак штити систем када обрада заостаје, а реплике убрзавају читање уз цену привремене застарелости. Сваки механизам има корист и цену коју тим мора умети да објасни.



Слика 1. Свесни компромиси када систем расте и ради под притиском

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

8.1 Конзистентност је одлука о гаранцији

- Фенцинг токен спречава да стари власник настави рад након преузимања.
- Повратни притисак бира контролисано успоравање или ограничен губитак пре потпуног колапса.
- Модел за читање може бити застарео, али систем мора показати колико и када то утиче на одлуку.

Питање за дискусију

За једну одлуку наведите да ли захтева најновије стање или може да користи застарео приказ, па објасните последицу избора.

8.2 Како се ова тема види у развоју

Завршна одбрана не тражи савршену архитектуру. Тражи јасан компромис: које стање мора бити тренутно, где је прихватљиво кашњење, како се спречава стари власник и шта систем ради када не може да прати улазни ритам.

Честе грешке које треба препознати

- Фаиловер покренут на основу једног кратког прекида.
- Неограничен ред порука који само одлаже квар.
- Коришћење застарелог реад модела као да је ауторитативно стање.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

8.3 Пример: два чвора и један ексклузиван посао

Две инстанце накратко изгубе везу једна са другом. Обе могу мислити да смеју да наставе исти ексклузиван посао. Фенцинг токен или епоха даје новом власнику јачи доказ власништва, па систем може одбити закашнели покушај старе инстанце чак и када се она касније поново појави.

- Координација бира једног важећег извршиоца за рад који не сме бити паралелан.
- Повратни притисак одлучује како систем реагује када улазни ритам надмаши обраду.
- Реплика убрзава читање, али њен приказ мора моћи да покаже застарелост.

Како се сценарио анализира

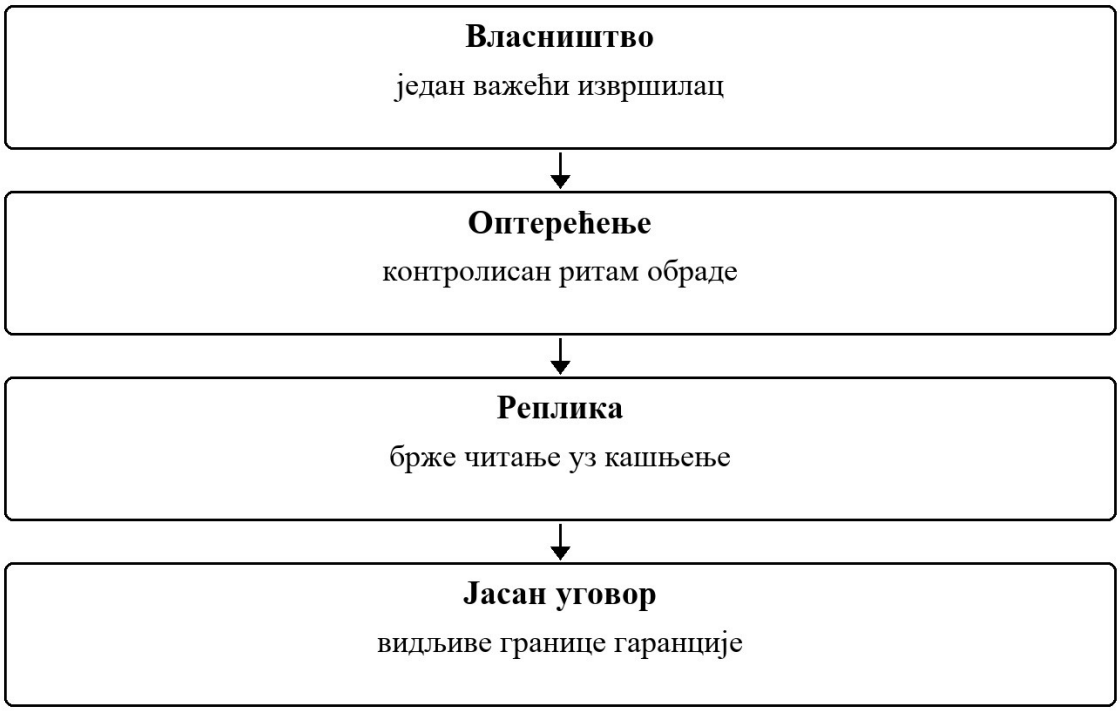
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

8.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

8.5 Шема рада под притиском



Слика 2. Шема рада под притиском

Напредни дистрибуирани дизајн не скрива компромис, већ кориснику и оператеру јасно показује гаранцију.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преносе и сценарио отказа који може променити ток.

8.6 Упоредни преглед елемената тока

Табела 1. Елемент и Улога у току

Елемент	Улога у току	Питање које треба поставити
Власништво	један важећи извршилац	Шта се дешава када власништво није доступан или даје застарелу информацију?
Оптерећење	ритам производње и обраде	Шта се дешава када оптерећење није доступан или даје застарелу информацију?
Реплика	брже читање уз кашњење	Шта се дешава када реплика није доступан или даје застарелу информацију?
Јасан уговор	видљива ограничења	Шта се дешава када јасан уговор није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

8.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Фенцинг токен спречава да стари власник настави рад након преузимања.

2. Повратни притисак бира контролисано успоравање или ограничен губитак пре потпуног колапса.

3. Модел за читање може бити застарео, али систем мора показати колико и када то утиче на одлуку.

4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

За једну одлуку наведите да ли захтева најновије стање или може да користи застарео приказ, па објасните последицу избора.

8.8 Извршиви пример

Пример показује две гаранције које систем мора да изабере када расте. Прва је власништво: само један воркер сме да мења распоред станице, власништво је временски ограничено (леасе), а ресурс одбија упис бившег власника по застарелом токену. Друга је проток: када је ред пун, пошиљалац добија јасан одговор уместо да посао неограничено чека.

Пример 1. Покретање демонстрације и тестова

```
cd examples/odp-dotnet/vezba-08-koordinacija-protok
dotnet run --project src/Odp.Vezba08.ConsoleUi
dotnet test Odp.Vezba08.sln
```

Правило у коду

Пример 2. examples/odp-dotnet/vezba-08-koordinacija-protok/src/Odp.Vezba08.Domain/Leases/LeasePolicy.cs

```
public LeaseDecision Acquire(Lease? current, string resource, string candidateId,
DateTimeOffset now)
{
    ArgumentException.ThrowIfNullOrWhiteSpace(resource);
    ArgumentException.ThrowIfNullOrWhiteSpace(candidateId);

    if (current is null || now >= current.ExpiresAt)
    {
        var token = (current?.FencingToken ?? 0) + 1;
        return new(true, LeaseCodes.Granted, new Lease(resource, candidateId, token, now
+ _duration));
    }

    return current.OwnerId == candidateId
        ? new(true, LeaseCodes.Renewed, current with { ExpiresAt = now + _duration })
        : new(false, LeaseCodes.HeldByAnother, current);
}
```

Тест који задаје квар

Пример 3. examples/odp-dotnet/vezba-08-koordinacija-protok/tests/Odp.Vezba08.Tests/Application/CoordinationFlowTests.cs

```
[Test]
public void Write_WhenFormerOwnerWakesUpAfterTakeover_IsRejectedByItsStaleToken()
{
    var former = _acquire.Acquire(Resource, "worker-a").Lease;
    _clock.Advance(TimeSpan.FromSeconds(45));
    var current = _acquire.Acquire(Resource, "worker-b").Lease;
    _write.Write(Resource, current.FencingToken, "plan B");

    var code = _write.Write(Resource, former.FencingToken, "zakasneli plan A");

    using (Assert.EnterMultipleScope())
    {
        Assert.That(code, Is.EqualTo(FencingCodes.StaleToken));
        Assert.That(_schedules.ValueOf(Resource), Is.EqualTo("plan B"));
    }
}
```

Испис демонстрације

Пример 4. дотнет рун --пројецт src/Odp.Vezba08.ConsoleUi

```
Koordinacija:
  worker-a traži lease -> lease_granted (vlasnik worker-a, token 1)
  worker-b traži lease -> lease_held_by_another (vlasnik worker-a, token 1)
  worker-a upisuje "plan A" sa tokenom 1 -> write_accepted
  worker-a je zastao 45 s; lease je istekao
  worker-b traži lease -> lease_granted (vlasnik worker-b, token 2)
  worker-b upisuje "plan B" sa tokenom 2 -> write_accepted
  worker-a upisuje "zakasneli plan A" sa tokenom 1 -> stale_fencing_token
  važeći raspored: plan B

Protok (kapacitet reda 2):
  job-1 -> job_accepted
  job-2 -> job_accepted
  job-3 -> queue_overloaded
```

Шта треба уочити

- У једном тренутку само један воркер је власник ресурса.
- После истека леасе-а нови власник добија већи токен, а упис бившег власника је одбијен.
- Када је ред пун, пошиљалац добија `queue_overloaded` уместо тихог чекања.

Задатак уз пример

Додати у приказ распореда податак о свежини: време последњег уписа и ознаку `IsStale` када је приказ старији од задатог прага. Тестом показати да читалац види застарео, али јасно означен податак док власник не упише нови.

Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Координација, проток и евентуална конзистентност". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.

П4 Оперативни и напредни ниво завршна одбрана

Контролна тачка

Ова тачка заокружује претходне теме. Организациони детаљи и захтеви налазе се у посебном одељку контролних тачака.